

GymPro

SQL Queries Reference

Complete documentation of all database queries used in the GymPro Membership Management System — with purpose, explanation and return values for every query.

Database	gym_membership_db
Tables	8 (MEMBERSHIP_PLAN, MEMBER, TRAINER, CLASS, ENROLLMENT, ATTENDANCE, PAYMENT, TRAINER_
Queries	50 documented queries across 9 categories
Engine	MySQL / MariaDB • InnoDB with Foreign Keys
Generated	08 September 2026

Table of Contents

1. Dashboard & Statistics Queries

Counts, revenue totals, expiring memberships, today's attendance

2. Membership Plan Queries

List, create, update, delete membership plans; count members per plan

3. Member Queries

Register, search, filter, update, delete members; plan assignment

4. Trainer Queries

Add, list, update, delete trainers; salary and class summaries

5. Class Queries

Schedule, list, filter, update, delete classes; capacity checks

6. Enrollment Queries

Enroll members, check duplicates, remove enrollments, list by class

7. Attendance Queries

Mark, update, delete attendance; calculate attendance rate per class

8. Payment Queries

Record membership fees, trainer salaries, filter payments, totals

9. Trainer Request Queries

Submit, accept, reject, delete trainer requests; status counts

1. Dashboard & Statistics Queries

These queries run every time the dashboard (index.php) loads. They provide live counts and summaries shown in the stat cards and quick-action panels.

Q1 Total Members Count

Purpose: Counts every row in the MEMBER table to display the total number of registered members in the top stat card.

Used in: index.php — "Total Members" stat card

SQL Query:

```
SELECT COUNT(*) AS n
FROM MEMBER
```

Returns: Single integer n — total number of members.

Q2 Active Members Count

Purpose: Counts members whose plan end date is today or in the future, meaning their membership is still valid and they have access to the gym.

Used in: index.php — subtitle of "Total Members" card

SQL Query:

```
SELECT COUNT(*) AS n
FROM MEMBER
WHERE PlanEndDate >= CURDATE()
```

Returns: Integer n — number of members with an active plan.

Q3 Total Revenue from Membership Fees

Purpose: Sums all payment amounts where the payment type is "Membership". COALESCE returns 0 instead of NULL when no payments exist yet.

Used in: index.php — "Revenue Collected" stat card

SQL Query:

```
SELECT COALESCE(SUM(Amount), 0) AS n
FROM PAYMENT
WHERE PaymentType = 'Membership'
```

Returns: Decimal n — total rupees collected from membership fees.

Q4 Total Scheduled Classes

Purpose: Counts all rows in the CLASS table, representing every class that has been scheduled in the system.

Used in: index.php — "Classes Scheduled" stat card

SQL Query:

```
SELECT COUNT(*) AS n
FROM CLASS
```

Returns: Integer n — total classes in the system.

Q5 Total Trainers Count

Purpose: Counts every trainer in the system, shown as a sub-label next to the classes stat.

Used in: index.php — subtitle of "Classes Scheduled" card

SQL Query:

```
SELECT COUNT(*) AS n
FROM TRAINER
```

Returns: Integer n — number of trainers on staff.

Q6 Today's Attendance Count

Purpose: Counts all attendance records for today where status is "Present". CURDATE() returns today's date dynamically without any hardcoded value.

Used in: index.php — "Today's Attendance" stat card

SQL Query:

```
SELECT COUNT(*) AS n
FROM ATTENDANCE
WHERE Date = CURDATE() AND Status = 'Present'
```

Returns: Integer n — number of members present today across all classes.

Q7 Pending Trainer Requests Count

Purpose: Counts all trainer requests that have not yet been accepted or rejected, alerting staff to review them.

Used in: index.php — "Trainer Requests Pending" sub-label

SQL Query:

```
SELECT COUNT(*) AS n
FROM TRAINER_REQUEST
WHERE Status = 'Pending'
```

Returns: Integer n — number of unreviewed trainer requests.

Q8 Recent Member Registrations

Purpose: Retrieves the six most recently registered members joined with their plan name. Uses LEFT JOIN so members without a plan still appear.

Used in: index.php — "Recent Registrations" dashboard panel

SQL Query:

```
SELECT m.Name, m.RegDate, p.PlanName
FROM MEMBER m
LEFT JOIN MEMBERSHIP_PLAN p ON m.PlanID = p.PlanID
ORDER BY m.MemberID DESC
LIMIT 6
```

Returns: Up to 6 rows: member name, registration date, plan name.

Q9 Upcoming Classes List

Purpose: Fetches classes from today onward, ordered by nearest date first. LEFT JOIN includes the trainer name, showing "—" if no trainer is assigned.

Used in: index.php — "Upcoming Classes" dashboard panel

SQL Query:

```
SELECT c.ClassName, c.DateOfClass, c.StartTime, t.Name AS TrainerName
FROM CLASS c
LEFT JOIN TRAINER t ON c.TrainerID = t.TrainerID
WHERE c.DateOfClass >= CURDATE()
ORDER BY c.DateOfClass ASC, c.StartTime ASC
LIMIT 6
```

Returns: Up to 6 upcoming classes with trainer names.

Q10 **Memberships Expiring in Next 7 Days**

Purpose: Finds members whose plan expires between today and 7 days from now. `DATE_ADD()` adds an interval to a date. Used to proactively alert staff to contact members for renewal.

Used in: `index.php` — "Expiring in 7 Days" warning panel

SQL Query:

```
SELECT Name, PlanEndDate
FROM MEMBER
WHERE PlanEndDate BETWEEN CURDATE()
      AND DATE_ADD(CURDATE(), INTERVAL 7 DAY)
ORDER BY PlanEndDate ASC
LIMIT 5
```

Returns: Up to 5 rows: member name and expiry date.

Note: `BETWEEN` is inclusive on both ends. Members expiring exactly today are included.

2. Membership Plan Queries

Queries that manage the MEMBERSHIP_PLAN table. Plans define the duration and cost of gym membership packages available to members.

Q11 List All Plans with Member Count

Purpose: Retrieves every membership plan along with a count of how many members are currently on each plan. Uses LEFT JOIN + GROUP BY so plans with zero members still appear in the list.

Used in: plans.php — main plan list table

SQL Query:

```
SELECT p.*, COUNT(m.MemberID) AS members
FROM MEMBERSHIP_PLAN p
LEFT JOIN MEMBER m ON m.PlanID = p.PlanID
GROUP BY p.PlanID
ORDER BY p.Price ASC
```

Returns: All plan columns + members count, sorted cheapest first.

Q12 Create New Membership Plan

Purpose: Inserts a new plan row. Duration is stored in months (e.g. 3 = quarterly). Price is a DECIMAL(10,2) for accurate currency storage.

Used in: plan_add.php — form submission

SQL Query:

```
INSERT INTO MEMBERSHIP_PLAN (PlanName, PlanDetails, Duration, Price)
VALUES ('Premium Annual', 'All access + PT sessions', 12, 22000.00)
```

Returns: Affected rows = 1 on success. New PlanID auto-generated.

Q13 Update Existing Plan

Purpose: Updates all editable fields of a plan identified by PlanID. After a plan price changes, existing members are unaffected (their payment history already recorded the old price).

Used in: plan_edit.php — form submission

SQL Query:

```
UPDATE MEMBERSHIP_PLAN
SET PlanName='Elite Annual', PlanDetails='Unlimited access',
    Duration=12, Price=25000.00
WHERE PlanID = 4
```

Returns: Affected rows = 1 if updated, 0 if PlanID not found.

Q14 Delete a Plan (Unlink Members First)

Purpose: Before deleting a plan, all members on that plan are unlinked by setting PlanID to NULL. Then the plan row is deleted. Two queries run in sequence.

Used in: plan_delete.php — confirmation form

SQL Query:

```
-- Step 1: Unlink members
UPDATE MEMBER
SET PlanID=NULL, PlanStartDate=NULL, PlanEndDate=NULL
WHERE PlanID = 4

-- Step 2: Delete the plan
DELETE FROM MEMBERSHIP_PLAN
WHERE PlanID = 4
```

Returns: Step 1: rows updated. Step 2: 1 row deleted.

Note: The ON DELETE SET NULL foreign key constraint also handles this automatically, but explicit unlinking is done for clarity.

Q15 Get Plan Duration (for Auto-calculating End Date)

Purpose: Fetches only the Duration field of a selected plan, used to compute $\text{PlanEndDate} = \text{PlanStartDate} + \text{Duration months}$ when assigning a plan to a member.

Used in: member_add.php, member_edit.php — plan end date calculation

SQL Query:

```
SELECT Duration
FROM MEMBERSHIP_PLAN
WHERE PlanID = 2
```

Returns: Single integer Duration (months).

3. Member Queries

Full CRUD plus search, filter, and profile-view queries for the MEMBER table. Member records are central to the system — nearly every other table links back here.

Q16 List Members with Search & Filter

Purpose: The main member list query. Supports three simultaneous filters: text search across Name/NIC/Email, plan filter, and active/expired status filter. Filters are appended dynamically in PHP to the WHERE clause.

Used in: members.php — member list with filters

SQL Query:

```
SELECT m.*, p.PlanName, p.Price
FROM MEMBER m
LEFT JOIN MEMBERSHIP_PLAN p ON m.PlanID = p.PlanID
WHERE m.Name LIKE '%kasun%'
      AND m.PlanID = 2
      AND m.PlanEndDate >= CURDATE()
ORDER BY m.MemberID DESC
```

Returns: Matching member rows with plan name and price.

Note: The actual WHERE clause is built dynamically — filters are added only if the user applied them.

Q17 Check NIC Uniqueness Before Insert

Purpose: Before saving a new member, this query checks whether the NIC already exists. NIC (National Identity Card) is a unique identifier in Sri Lanka.

Used in: member_add.php — pre-insert validation

SQL Query:

```
SELECT MemberID
FROM MEMBER
WHERE NIC = '199512345678'
```

Returns: Row if NIC exists (duplicate), empty result if unique.

Q18 Insert New Member

Purpose: Registers a new member. Age is auto-calculated from DOB in PHP. PlanEndDate is computed as PlanStartDate + Duration months. NULL values are used for optional fields.

Used in: member_add.php — form submission

SQL Query:

```
INSERT INTO MEMBER
(Name, NIC, DOB, Age, Gender, Email, Address,
 PhoneNo, RegDate, PlanID, PlanStartDate, PlanEndDate)
VALUES
('Kasun Fernando', '199512345678', '1995-06-15',
 29, 'Male', 'kasun@example.com', '12 Main St',
 '0712345678', '2025-01-10', 4, '2025-01-10', '2026-01-10')
```

Returns: Affected rows = 1. New MemberID auto-incremented.

Q19 Update Member Record

Purpose: Updates all fields of a member by MemberID. If the plan changes, PlanEndDate is recalculated. If plan is removed, PlanID/dates are set to NULL.

Used in: member_edit.php — form submission

SQL Query:

```
UPDATE MEMBER SET
Name='Kasun Fernando', NIC='199512345678',
DOB='1995-06-15', Age=29, Gender='Male',
Email='kasun@example.com', Address='12 Main St',
PhoneNo='0712345678', RegDate='2025-01-10',
PlanID=4, PlanStartDate='2025-01-10', PlanEndDate='2026-01-10'
WHERE MemberID = 1
```

Returns: Affected rows = 1 if updated.

Q20 Delete Member (Cascade All Related Records)

Purpose: Deletes a member and all their associated records in the correct dependency order to avoid foreign key constraint violations. The CASCADE setting handles this automatically, but explicit deletes are also performed.

Used in: member_delete.php — confirmation form

SQL Query:

```
DELETE FROM ENROLLMENT WHERE MemberID = 1;
DELETE FROM ATTENDANCE WHERE MemberID = 1;
DELETE FROM PAYMENT WHERE MemberID = 1;
DELETE FROM TRAINER_REQUEST WHERE MemberID = 1;
DELETE FROM MEMBER WHERE MemberID = 1;
```

Returns: Each query returns affected row count.

Note: ON DELETE CASCADE on foreign keys would handle child rows automatically, but explicit deletes are run for safety.

Q21 Member Profile — Full Detail View

Purpose: Fetches a single member with their full plan details for the profile page.

Used in: member_view.php — profile page header

SQL Query:

```
SELECT m.*, p.PlanName, p.Duration, p.Price
FROM MEMBER m
LEFT JOIN MEMBERSHIP_PLAN p ON m.PlanID = p.PlanID
WHERE m.MemberID = 1
```

Returns: Single row: all member fields + plan name, duration, price.

Q22 Member Profile — Enrolled Classes

Purpose: Lists all classes a specific member is enrolled in, via the ENROLLMENT junction table.

Used in: member_view.php — enrolled classes panel

SQL Query:

```
SELECT c.ClassName, c.DateOfClass, c.StartTime
FROM ENROLLMENT e
JOIN CLASS c ON e.ClassID = c.ClassID
WHERE e.MemberID = 1
ORDER BY c.DateOfClass DESC
```

Returns: Class name, date, and start time for each enrolled class.

4. Trainer Queries

Queries for managing gym trainers. Trainers are linked to classes (via CLASS.TrainerID) and salary payments (via PAYMENT.TrainerID).

Q23 List All Trainers with Class Count & Salary Paid

Purpose: Fetches every trainer with two aggregates: how many classes they teach (COUNT DISTINCT to avoid duplicates from the payment join) and total salary paid.

Used in: trainers.php — trainer list table

SQL Query:

```
SELECT t.*,  
       COUNT(DISTINCT c.ClassID) AS class_count,  
       COALESCE(SUM(p.Amount), 0) AS salary_paid  
FROM TRAINER t  
LEFT JOIN CLASS c ON c.TrainerID = t.TrainerID  
LEFT JOIN PAYMENT p ON p.TrainerID = t.TrainerID AND p.PaymentType='Salary'  
GROUP BY t.TrainerID  
ORDER BY t.TrainerID DESC
```

Returns: All trainer columns + class_count + salary_paid.

Note: COUNT DISTINCT is essential here — without it, joining two LEFT JOINS would multiply rows and inflate the count.

Q24 Insert New Trainer

Purpose: Adds a new trainer. Experience is stored in years as an integer.

Used in: trainer_add.php — form submission

SQL Query:

```
INSERT INTO TRAINER (Name, PhoneNo, Email, Experience)  
VALUES ('Ashan Perera', '0771234567', 'ashan@gym.lk', 7)
```

Returns: Affected rows = 1. New TrainerID auto-generated.

Q25 Auto-calculate Trainer Salary from Classes

Purpose: Before showing the payment form, this query sums up RatePerClass for each trainer across all their classes. This auto-fills the Amount field when recording a salary payment.

Used in: payment_add.php — auto-fill salary amount

SQL Query:

```
SELECT t.TrainerID, t.Name,  
       COALESCE(SUM(c.RatePerClass), 0) AS calc_salary  
FROM TRAINER t  
LEFT JOIN CLASS c ON c.TrainerID = t.TrainerID  
WHERE c.RatePerClass IS NOT NULL  
GROUP BY t.TrainerID
```

Returns: TrainerID, Name, calculated total salary from class rates.

Q26 Delete Trainer (Unlink Classes & Payments First)

Purpose: Removes a trainer safely by first clearing their TrainerID from related records, then deleting their requests and finally the trainer row itself.

Used in: trainer_delete.php — confirmation form

SQL Query:

```
DELETE FROM TRAINER_REQUEST WHERE TrainerID = 1;  
UPDATE CLASS SET TrainerID = NULL WHERE TrainerID = 1;  
UPDATE PAYMENT SET TrainerID = NULL WHERE TrainerID = 1;  
DELETE FROM TRAINER WHERE TrainerID = 1;
```

Returns: Each query returns affected row count.

5. Class Queries

Classes are scheduled gym sessions tied to a trainer and a date/time. Capacity is enforced at the enrollment stage.

Q27 List All Classes with Trainer & Enrollment Count

Purpose: The main class list. Counts enrolled members per class and compares to Capacity. Uses GROUP BY with COUNT DISTINCT to avoid join inflation.

Used in: classes.php — class list table

SQL Query:

```
SELECT c.*, t.Name AS TrainerName,
       COUNT(DISTINCT e.EnrollmentID) AS enrolled
FROM CLASS c
LEFT JOIN TRAINER t ON c.TrainerID = t.TrainerID
LEFT JOIN ENROLLMENT e ON e.ClassID = c.ClassID
GROUP BY c.ClassID
ORDER BY c.DateOfClass DESC, c.StartTime ASC
```

Returns: All class columns + trainer name + enrollment count.

Q28 List Classes with Available Spots (for Enrollment)

Purpose: When enrolling a member, only classes that still have capacity are shown. HAVING filters groups after aggregation — it cannot be used in WHERE.

Used in: enrollment_add.php — class dropdown

SQL Query:

```
SELECT c.ClassID, c.ClassName, c.DateOfClass, c.StartTime,
       c.Capacity, COUNT(e.EnrollmentID) AS enrolled
FROM CLASS c
LEFT JOIN ENROLLMENT e ON e.ClassID = c.ClassID
GROUP BY c.ClassID
HAVING enrolled < c.Capacity
ORDER BY c.DateOfClass DESC
```

Returns: Only classes where enrolled count is less than capacity.

Note: HAVING is used (not WHERE) because the filter is on an aggregate value (COUNT).

Q29 Insert New Class

Purpose: Schedules a new class. TrainerID and RatePerClass are nullable — a class can exist without a trainer initially.

Used in: class_add.php — form submission

SQL Query:

```
INSERT INTO CLASS
  (ClassName, DateOfClass, StartTime, EndTime,
   Description, Capacity, TrainerID, Specialization, RatePerClass)
VALUES
  ('Morning Yoga', '2025-09-10', '06:00', '07:00',
   'Beginner session', 15, 1, 'Yoga', 1500.00)
```

Returns: Affected rows = 1. ClassID auto-generated.

Q30 Delete Class (Cascade Enrollments & Attendance)

Purpose: Removes a class and all linked enrollment and attendance records to maintain referential integrity.

Used in: class_delete.php — confirmation form

SQL Query:

```
DELETE FROM ENROLLMENT WHERE ClassID = 1;
DELETE FROM ATTENDANCE WHERE ClassID = 1;
DELETE FROM CLASS      WHERE ClassID = 1;
```

Returns: Each statement returns affected row count.

6. Enrollment Queries

ENROLLMENT is the junction table linking MEMBER and CLASS in a many-to-many relationship. A member can be in many classes, and a class has many members.

Q31 List All Enrollments (with Search)

Purpose: Joins ENROLLMENT with MEMBER and CLASS to show meaningful names instead of raw IDs. Supports an optional name/class search.

Used in: enrollments.php — enrollment list

SQL Query:

```
SELECT e.EnrollmentID, m.Name AS MemberName, m.MemberID,
       c.ClassName, c.DateOfClass, c.StartTime, c.ClassID
FROM ENROLLMENT e
JOIN MEMBER m ON e.MemberID = m.MemberID
JOIN CLASS c ON e.ClassID = c.ClassID
WHERE m.Name LIKE '%kasun%'
ORDER BY c.DateOfClass DESC, m.Name ASC
```

Returns: EnrollmentID, member name, class name, date, time.

Q32 Check for Duplicate Enrollment

Purpose: Before inserting a new enrollment, this verifies the member is not already enrolled in the same class. Prevents duplicate ENROLLMENT rows.

Used in: enrollment_add.php — pre-insert check

SQL Query:

```
SELECT EnrollmentID
FROM ENROLLMENT
WHERE MemberID = 1 AND ClassID = 2
```

Returns: Row if duplicate exists, empty result if enrollment is valid.

Q33 **Insert New Enrollment**

Purpose: Adds a member to a class. The combination of MemberID + ClassID uniquely identifies the enrollment.

Used in: enrollment_add.php — form submission

SQL Query:

```
INSERT INTO ENROLLMENT (MemberID, ClassID)
VALUES (1, 2)
```

Returns: Affected rows = 1. EnrollmentID auto-generated.

Q34 **Delete (Remove) an Enrollment**

Purpose: Unenrolls a member from a class by deleting the specific ENROLLMENT row.

Used in: enrollment_delete.php — confirmation form

SQL Query:

```
DELETE FROM ENROLLMENT
WHERE EnrollmentID = 5
```

Returns: Affected rows = 1.

7. Attendance Queries

Attendance records link a member, a class, and a date with a Present/Absent status. The mark-attendance page uses an upsert pattern to update if a record exists or insert if not.

Q35 List Attendance with Filters

Purpose: Fetches attendance records with optional filters for date, class, and status. Two separate WHERE strings are built — one with table alias for the JOIN query, one without alias for plain COUNT queries.

Used in: attendance.php — attendance list with filters

SQL Query:

```
SELECT a.*, m.Name AS MemberName, c.ClassName
FROM ATTENDANCE a
JOIN MEMBER m ON a.MemberID = m.MemberID
JOIN CLASS c ON a.ClassID = c.ClassID
WHERE a.Date = '2025-09-08'
      AND a.ClassID = 1
      AND a.Status = 'Present'
ORDER BY a.Date DESC, m.Name ASC
```

Returns: Matching attendance rows with member and class names.

Q36 Count Total & Present Attendance (for Rate)

Purpose: Two plain COUNT queries used to calculate the attendance rate percentage shown above the table. Uses column names without alias (no JOIN needed).

Used in: attendance.php — attendance rate display

SQL Query:

```
-- Total records matching the filter
SELECT COUNT(*) AS n
FROM ATTENDANCE
WHERE Date = '2025-09-08' AND ClassID = 1

-- Present count (adds Status filter)
SELECT COUNT(*) AS n
FROM ATTENDANCE
WHERE Date = '2025-09-08' AND ClassID = 1 AND Status = 'Present'
```

Returns: Two integers: total records, and present count.

*Note: Rate = present / total * 100. Division by zero is guarded by checking total > 0 in PHP.*

Q37 Load Members for Attendance Marking

Purpose: When marking attendance for a class, retrieves all enrolled members plus their existing attendance status for that class on the given date (if any). A subquery checks the ATTENDANCE table inline.

Used in: attendance_mark.php — member list with radio buttons

SQL Query:

```
SELECT m.MemberID, m.Name,
       (SELECT Status FROM ATTENDANCE
        WHERE MemberID = m.MemberID
          AND ClassID = 1
          AND Date = '2025-09-08') AS status
FROM ENROLLMENT e
JOIN MEMBER m ON e.MemberID = m.MemberID
WHERE e.ClassID = 1
ORDER BY m.Name
```

Returns: Member ID, name, and existing attendance status (NULL if not yet marked).

Note: The correlated subquery returns NULL when no attendance record exists — PHP treats this as "not yet marked" and defaults the radio to Present.

Q38 Upsert Attendance (Insert or Update)

Purpose: For each member in the class, checks if an attendance record already exists. If it does, UPDATE is used. If not, INSERT is used. This is the upsert pattern.

Used in: attendance_mark.php — form submission loop

SQL Query:

```
-- Check if record exists
SELECT AttendanceID FROM ATTENDANCE
WHERE MemberID=1 AND ClassID=1 AND Date='2025-09-08'

-- If exists: update
UPDATE ATTENDANCE SET Status='Present'
WHERE AttendanceID = 7

-- If not: insert
INSERT INTO ATTENDANCE (Status, Date, MemberID, ClassID)
VALUES ('Present', '2025-09-08', 1, 1)
```

Returns: 1 affected row per member per call.

Note: This allows re-marking attendance for the same session without creating duplicate records.

8. Payment Queries

The PAYMENT table serves dual purpose: recording membership fees paid by members and salary payments made to trainers. PaymentType distinguishes the two.

Q39 List Payments with Filters

Purpose: Retrieves payments with optional filters for type (Membership/Salary), payment method, and date range. Uses aliased WHERE for the JOIN query.

Used in: payments.php — payment list with filters

SQL Query:

```
SELECT p.*, m.Name AS MemberName, t.Name AS TrainerName
FROM PAYMENT p
LEFT JOIN MEMBER m ON p.MemberID = m.MemberID
LEFT JOIN TRAINER t ON p.TrainerID = t.TrainerID
WHERE p.PaymentType = 'Membership'
      AND p.PaymentMethod = 'Cash'
      AND p.PaymentDate >= '2025-01-01'
      AND p.PaymentDate <= '2025-12-31'
ORDER BY p.PaymentDate DESC, p.PaymentID DESC
```

Returns: Payment rows with member/trainer names where applicable.

Q40 Payment Totals (Membership, Salary, Grand Total)

Purpose: A single query using CASE WHEN to split the SUM into membership vs salary, plus a grand total. Uses plain WHERE (no alias) since it is a single-table query.

Used in: payments.php — three summary stat cards

SQL Query:

```
SELECT
  SUM(CASE WHEN PaymentType='Membership' THEN Amount ELSE 0 END) AS membership,
  SUM(CASE WHEN PaymentType='Salary' THEN Amount ELSE 0 END) AS salary,
  SUM(Amount) AS total
FROM PAYMENT
WHERE PaymentType = 'Membership'
      AND PaymentDate >= '2025-01-01'
```

Returns: Three decimal values: membership total, salary total, grand total.

Note: CASE WHEN inside SUM is a conditional aggregate — it sums only the rows matching that condition.

Q41 Insert Membership Payment

Purpose: Records a fee paid by a member. TrainerID is NULL for membership payments.

Used in: payment_add.php — membership fee form

SQL Query:

```
INSERT INTO PAYMENT
  (PaymentDate, PaymentMethod, Amount, PaymentType, MemberID, TrainerID)
VALUES ('2025-09-08', 'Cash', 22000.00, 'Membership', 1, NULL)
```

Returns: Affected rows = 1. PaymentID auto-generated.

Q42 Insert Trainer Salary Payment

Purpose: Records a salary payment to a trainer. MemberID is NULL for salary payments.

Used in: payment_add.php — salary form

SQL Query:

```
INSERT INTO PAYMENT
  (PaymentDate, PaymentMethod, Amount, PaymentType, MemberID, TrainerID)
VALUES ('2025-08-31', 'Bank Transfer', 45000.00, 'Salary', NULL, 1)
```

Returns: Affected rows = 1.

Note: The PaymentType column distinguishes membership fees from salary entries, allowing both to share the same table.

9. Trainer Request Queries

Members can request a specific trainer. Staff can accept or reject requests. Status values are: Pending, Accepted, Rejected.

Q43 List All Requests with Status Filter

Purpose: Fetches all trainer requests with member and trainer names. Optional status filter allows viewing only Pending, Accepted, or Rejected.

Used in: `trainer_requests.php` — main request table

SQL Query:

```
SELECT tr.*, m.Name AS MemberName, t.Name AS TrainerName
FROM TRAINER_REQUEST tr
JOIN MEMBER m ON tr.MemberID = m.MemberID
JOIN TRAINER t ON tr.TrainerID = t.TrainerID
WHERE tr.Status = 'Pending'
ORDER BY tr.RequestDate DESC, tr.RequestID DESC
```

Returns: Request rows with member and trainer names.

Q44 Count Requests by Status

Purpose: Counts requests grouped by their Status column. Returns one row per status value. Used to display the three mini-stat badges at the top of the page.

Used in: `trainer_requests.php` — Pending / Accepted / Rejected badges

SQL Query:

```
SELECT Status, COUNT(*) AS n
FROM TRAINER_REQUEST
GROUP BY Status
```

Returns: Up to 3 rows: one each for 'Pending', 'Accepted', 'Rejected'.

Q45 Insert New Trainer Request

Purpose: Submits a new request from a member for a specific trainer. Status defaults to "Pending".

Used in: request_add.php — form submission

SQL Query:

```
INSERT INTO TRAINER_REQUEST (RequestDate, Status, MemberID, TrainerID)
VALUES ('2025-09-08', 'Pending', 3, 1)
```

Returns: Affected rows = 1. RequestID auto-generated.

Q46 Accept or Reject a Request

Purpose: A single UPDATE changes the Status of a request. The \$status variable in PHP is validated to only allow "Accepted" or "Rejected" before the query runs.

Used in: request_update.php — one-click Accept / Reject buttons

SQL Query:

```
UPDATE TRAINER_REQUEST
SET Status = 'Accepted'
WHERE RequestID = 3
```

Returns: Affected rows = 1 if updated.

Note: PHP validates \$status ∈ {'Accepted','Rejected'} before running this query to prevent arbitrary status values.

Q47 Delete a Trainer Request

Purpose: Permanently removes a request record regardless of its status.

Used in: request_delete.php — confirmation form

SQL Query:

```
DELETE FROM TRAINER_REQUEST
WHERE RequestID = 3
```

Returns: Affected rows = 1.

Bonus: Setup & Admin Queries

These queries are used by setup.php and seed.php for initial database creation and data seeding. They run once during system setup, not in normal operation.

Q48 Create the Database

Purpose: Creates the database if it does not already exist. IF NOT EXISTS prevents an error when run multiple times.

Used in: setup.php, seed.php

SQL Query:

```
CREATE DATABASE IF NOT EXISTS gym_membership_db
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci
```

Returns: Query OK.

Note: utf8mb4 supports full Unicode including emoji. Always prefer utf8mb4 over utf8 in MySQL.

Q49 Verify Foreign Key Constraints

Purpose: Queries the information_schema to confirm that all foreign keys are correctly set up after table creation. Used in the diagnostic report in seed.php.

Used in: seed.php — FK verification step

SQL Query:

```
SELECT TABLE_NAME, COLUMN_NAME,
       REFERENCED_TABLE_NAME, REFERENCED_COLUMN_NAME
FROM information_schema.KEY_COLUMN_USAGE
WHERE REFERENCED_TABLE_SCHEMA = 'gym_membership_db'
ORDER BY TABLE_NAME
```

Returns: One row per foreign key: table, column, referenced table and column.

Q50 Truncate All Tables for Re-seeding

Purpose: Clears all data from every table in dependency-safe order. FOREIGN_KEY_CHECKS=0 temporarily disables FK enforcement to allow TRUNCATE in any order.

Used in: seed.php — re-seed wipe step

SQL Query:

```
SET FOREIGN_KEY_CHECKS = 0;
TRUNCATE TABLE TRAINER_REQUEST;
TRUNCATE TABLE PAYMENT;
TRUNCATE TABLE ATTENDANCE;
TRUNCATE TABLE ENROLLMENT;
TRUNCATE TABLE CLASS;
TRUNCATE TABLE MEMBER;
TRUNCATE TABLE TRAINER;
TRUNCATE TABLE MEMBERSHIP_PLAN;
SET FOREIGN_KEY_CHECKS = 1;
```

Returns: Query OK for each statement.

Note: TRUNCATE is faster than DELETE for full table clears — it resets AUTO_INCREMENT counters too. Only use during development/testing.

Query Summary

#	Query Name	Type	Table(s)	Used In
1	Total Members Count	READ	MEMBER	index.php
2	Active Members Count	READ	MEMBER	index.php
3	Total Revenue	READ	PAYMENT	index.php
4	Total Classes	READ	CLASS	index.php
5	Total Trainers	READ	TRAINER	index.php
6	Today's Attendance	READ	ATTENDANCE	index.php
7	Pending Requests Count	READ	TRAINER_REQUEST	index.php
8	Recent Registrations	READ	MEMBER + PLAN	index.php
9	Upcoming Classes	READ	CLASS + TRAINER	index.php
10	Expiring in 7 Days	READ	MEMBER	index.php
11	Plans with Member Count	READ	MEMBERSHIP_PLAN + MEMBER	plans.php
12	Create Plan	CREATE	MEMBERSHIP_PLAN	plan_add.php
13	Update Plan	UPDATE	MEMBERSHIP_PLAN	plan_edit.php
14	Delete Plan	DELETE	MEMBER + MEMBERSHIP_PLAN	plan_delete.php
15	Get Plan Duration	READ	MEMBERSHIP_PLAN	member_add/edit.php
16	List Members (Filtered)	READ	MEMBER + PLAN	members.php
17	NIC Uniqueness Check	VALIDATE	MEMBER	member_add.php
18	Insert Member	CREATE	MEMBER	member_add.php
19	Update Member	UPDATE	MEMBER	member_edit.php
20	Delete Member (Cascade)	DELETE	MEMBER + 4 tables	member_delete.php
21	Member Full Profile	READ	MEMBER + PLAN	member_view.php
22	Member Enrolled Classes	READ	ENROLLMENT + CLASS	member_view.php
23	Trainers with Aggregates	READ	TRAINER + CLASS + PAYMENT	trainers.php
24	Insert Trainer	CREATE	TRAINER	trainer_add.php
25	Auto-calculate Salary	READ	TRAINER + CLASS	payment_add.php
26	Delete Trainer	DELETE	TRAINER + 3 tables	trainer_delete.php
27	List Classes with Enrolled Count	READ	CLASS + TRAINER + ENROLLMENT	classes.php
28	Classes with Available Spots	READ	CLASS + ENROLLMENT	enrollment_add.php
29	Insert Class	CREATE	CLASS	class_add.php
30	Delete Class	DELETE	CLASS + ENROLLMENT + ATTENDANCE	class_delete.php

#	Query Name	Type	Table(s)	Used In
31	List Enrollments	READ	ENROLLMENT + MEMBER + CLASS	enrollments.php
32	Duplicate Enrollment Check	VALIDATE	ENROLLMENT	enrollment_add.php
33	Insert Enrollment	CREATE	ENROLLMENT	enrollment_add.php
34	Delete Enrollment	DELETE	ENROLLMENT	enrollment_delete.php
35	List Attendance (Filtered)	READ	ATTENDANCE + MEMBER + CLASS	attendance.php
36	Attendance Rate Counts	READ	ATTENDANCE	attendance.php
37	Load Members for Marking	READ	ENROLLMENT + MEMBER + ATTENDANCE	attendance_mark.php
38	Upsert Attendance	UPSERT	ATTENDANCE	attendance_mark.php
39	List Payments (Filtered)	READ	PAYMENT + MEMBER + TRAINER	payments.php
40	Payment Totals by Type	READ	PAYMENT	payments.php
41	Insert Membership Payment	CREATE	PAYMENT	payment_add.php
42	Insert Salary Payment	CREATE	PAYMENT	payment_add.php
43	List Requests (Filtered)	READ	TRAINER_REQUEST + MEMBER + TRAINER	trainer_requests.php
44	Count Requests by Status	READ	TRAINER_REQUEST	trainer_requests.php
45	Insert Trainer Request	CREATE	TRAINER_REQUEST	request_add.php
46	Accept / Reject Request	UPDATE	TRAINER_REQUEST	request_update.php
47	Delete Request	DELETE	TRAINER_REQUEST	request_delete.php
48	Create Database	SETUP	—	setup.php
49	Verify Foreign Keys	DIAGNOSTIC	information_schema	seed.php
50	Truncate All Tables	SETUP	All 8 tables	seed.php